

Código JS – variables constantes

Para crear variables constantes por bloque

El valor de una constante no puede cambiarse a través de la reasignación. Las constantes no se pueden redeclarar.

Este nombre es el nombre de rol, va en cada pagina

```
const usuarios = {  
  'Gerente': {password: '123Gerente456', rol: 'Gerente'},  
  'Administrador': {password: 'admin123', rol: 'Administrador'},  
  'Entrenador': {password: 'entrena123', rol: 'Entrenador'},  
  'Usuario': {password: 'usuario123', rol: 'Usuario'}  
};
```

Nombre
del
Usuario

Sintaxis

```
const varname1 = value1 [, varname2 = value2 [, varname3 = value3 [, ... [, varnameN =  
valueN]]]];
```

Permite cargar el Script dentro del body según la función VerificarAcceso

```
<body onload="verificarAcceso(['Gerente','Administrador','Entrenador','Usuario'])">
```

*esto va en el body de la pagina html que quieres que funcione

No permite capturar los valores de los inputs

Usuario
Contraseña
Ingresar

```
function validarLogin(){
    const usuario = document.getElementById('usuario').value;
    const password = document.getElementById('password').value;

    if (usuarios[usuario] && usuarios[usuario].password ===
password) {
        sessionStorage.setItem('usuario', usuario);
        sessionStorage.setItem('rol', usuarios[usuario].rol);
        document.getElementById('mensaje').innerText = 'Ingreso
exitoso';
        window.location = 'pagina1.html';
    } else {
        document.getElementById('mensaje').innerText = 'Credenciales
incorrectas.';
    }
}
```

Nuevo código (Guarda las contraseñas)

Cuadro para Mostrar login, html, css y script

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
</head>
<body>

  <style>
    /*Aca puedes escoger colocar el tipo de letra que se usara en el login o todo el
documento*/
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
      font-family: Verdana, Geneva, Tahoma, sans-serif;
    }

    /*Coloca el login en un recuadro en el centro*/
    .modal {
      position: fixed;
      top: 50%;
      left: 50%;
      transform: translate(-50%, -50%);
      background: white; /*Color de la caja*/
      padding: 30px;
      border-radius: 10px;
      z-index: 1001;
      box-shadow: 0 0 15px rgba(0,0,0,0.5);
      width: 300px; /*Ancho caja login*/
      text-align: center;
    }

    /*Los texfield del login tamaño*/
    .modal input {
      display: block;
      width: 95%; /*Ancho de los textField*/
      padding: 10px 0px ;
      margin: 10px ;
    }

    /*boton del textfiel*/
    .modal button {
      background: red;
      color: white;
      border: none;
      padding: 10px;
      width: 100%;
      cursor: pointer;
      border-radius: 5px;
    }

    /*Para la x de cerrar del login*/
```

```

.modal .close {
  position: absolute;
  top: 10px;
  right: 15px;
  cursor: pointer;
  font-size: 20px;
  color: #333;
}

.overlay {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 100%;
  backdrop-filter: blur(5px);
  background-color: rgba(0, 0, 0, 0.4);
  z-index: 1000;
}

```

```

/* Ocultar elementos */
.hidden {
  display: none;
}

```

```

</style>
<a href="#" id="login-btn">Login</a>
<div id="login-modal" class="modal hidden">
  <div class="modal-content">
    <span class="close">&times;</span>
    <h2>Iniciar Sesión</h2>
    <input type="text" placeholder="Usuario" id="username">
    <input type="password" placeholder="Contraseña" id="password">
    <button onclick="validarLogin()">Ingresar</button>
    <p id="mensaje"></p>
  </div>
</div>

```

```

<script>
//abrir el cuadro de login
document.getElementById('login-btn').addEventListener('click', function (e) {
  e.preventDefault();
  document.getElementById('login-modal').classList.remove('hidden');
  document.getElementById('overlay').classList.remove('hidden');
});

// cerrar el cuadro de login
document.querySelector('.close').addEventListener('click', function () {
  document.getElementById('login-modal').classList.add('hidden');
  document.getElementById('overlay').classList.add('hidden');
});

<!--Importante para llamar al difuminado de fondo del login -->
<div id="overlay" class="overlay hidden"></div>

</script>
</body>
</html>

```

Para mostrar el login y ocultarlo todo se hace desde script y esta línea importante de estilo

Símbolo de por "x" en HTML

Cuando se usa un botón en html, el navegador envía el formulario y recarga la página, esta función evita eso, maneja la acción con JavaScript

Busca elementos por el nombre id

Es un método. Escucha un evento específico que ocurrirá en ese elemento, en este caso "click"

Devuelve una lista de clases CSS que tienen un elemento

Es un método, que permite seleccionar el primer elemento dentro de html que tenga una clase de CSS con un nombre x, en este caso la clase CSS ".close"

En si todo este bloque de if solo se ejecuta, cuando dentro del localStorage no existe un elemento con ese nombre "usuarios", entonces en ese caso procede a crear ese elemento mediante el método const

Es un pequeño almacén de tu navegador donde tu puedes guardar variables, estas permanecen ahí incluso si tu cierras las pestañas

Usuarios por defecto y validar contraseñas

```
//***** CREAR USUARIOS POR DEFECTO***** */
```

```
// Inicializar usuarios si no existen
```

```
if (!localStorage.getItem("usuarios")) {  
  const usuariosIniciales = [  
    { username: "gerente", password: "1234", rol: "Gerente" },  
    { username: "admin", password: "admin1234", rol: "Administrador" },  
    { username: "entrenador", password: "entrenador1234", rol: "Entrenador" },  
    { username: "usuario", password: "usuario1234", rol: "Usuario" }  
  ];  
  localStorage.setItem("usuarios", JSON.stringify(usuariosIniciales));  
}
```

SetItem: Permite guardar cadenas en el localStorage

Como lo que vamos a guardar no es cadenas JSON.stringify permite convertirlo a cadena para guardar en el localStorage

!, para evaluar el if en True, existe un elemento usuarios → no, con el (!) lo vuelve "true" para ejecutar

```
//***** VALIDAR EL LOGIN***** */
```

```
// Validar login y redirigir según rol
```

```
function validarLogin() {  
  const username = document.getElementById('username').value.trim();  
  const password = document.getElementById('password').value.trim();  
  const mensaje = document.getElementById('mensaje');
```

Value: para ingresar al valor en ese campo de texto

Trim(): elimina los espacios en blanco del texto de ese campo de texto

Convierte a array las cadenas de texto almacenados en localStorage

```
  const usuarios = JSON.parse(localStorage.getItem("usuarios")) || [];  
  const usuarioEncontrado = usuarios.find(  
    user => user.username === username && user.password === password  
  );
```

Ó: si no encuentra un elemento así va a bota null

Find(): método para recorrer arrays y ejecutar una función de dentro los paréntesis, bota true o undefined

Función de tipo flecha

```
  if (usuarioEncontrado) {  
    // Guardar sesión actual  
    localStorage.setItem("usuarioActual", JSON.stringify(usuarioEncontrado));  
    mensaje.innerText = "Ingreso exitoso";
```

```
    // Redireccionar según el rol
```

```
    setTimeout(() => {  
      switch (usuarioEncontrado.rol) {  
        case "Usuario":  
          window.location.href = "usuario.html";  
          break;  
        case "Entrenador":  
          window.location.href = "entrenador.html";  
          break;  
        case "Administrador":  
          window.location.href = "admi.html";  
          break;  
        case "Gerente":  
          window.location.href = "gerente.html";  
          break;  
        default:  
          window.location.href = "index.html";  
      }  
    }, 1000); // Esperar 1 segundo para mostrar el mensaje
```

setTimeout, ejecuta líneas de código después de un determinado tiempo, en este caso 1 segundo

```
setTimeout(() => { ... }, 1000);
```

```
  } else {  
    mensaje.innerText = "Usuario o contraseña incorrectos."  
  }  
}
```

```
}
```